

# Virtualização

## VMs, Containers e Docker

# Contexto Histórico

- **Mainframes**

- Grandes computadores centrais acedidos por via de terminais simples (dumb).
- Partilha de recursos do mainframe, permitindo uma utilização mais eficiente do hardware central.

- **Servidores**

- Utilizados diretamente para executar aplicações ou em processamento em lote (batch).

- **Máquinas Pessoais**

- Menor necessidade de otimização intensiva de recursos
- Possibilidade de simular diferentes ambientes
  - e.g., Linux numa máquina Windows

# Definição

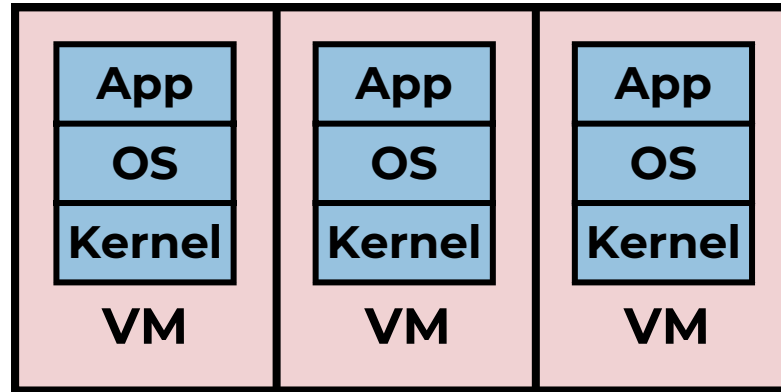
- Virtualização
  - Tecnologia que permite executar "ambientes virtuais" sobre uma única máquina física
  - Ambientes isolados e independentes para melhor aproveitamento dos recursos de hardware.
- Virtualização de Hardware
  - Full Virtualization
  - Hypervisores Tipo 1 (Bare-metal)
  - Hypervisores Tipo 2 (Hosted)

# Características

- Único host físico → múltiplas máquinas virtuais.
- Operam de forma independente
  - Configurações e falhas não afetem as outras.
- Adicionar ou remover VMs conforme a demanda.
- Inicialização lenta
  - Simulação de um sistema operacional completo.
- Requer monitoramento e manutenção
  - Aumento da complexidade operacional.

# Virtual Machines (VMs)

## Tipo 2

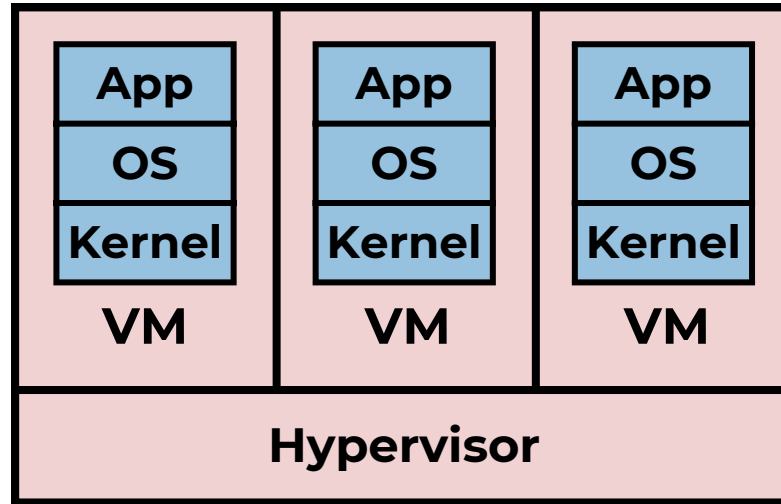


Exemplos: VMware, VirtualBox, KVM.

Figura inspirada em 'Monolith to Microservices'

# Virtual Machines (VMs)

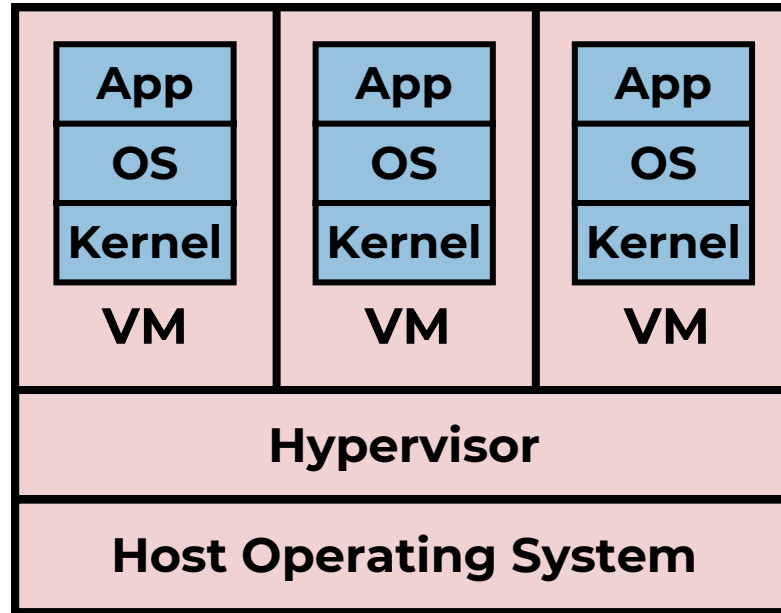
## Tipo 2



Exemplos: VMware, VirtualBox, KVM.

# Virtual Machines (VMs)

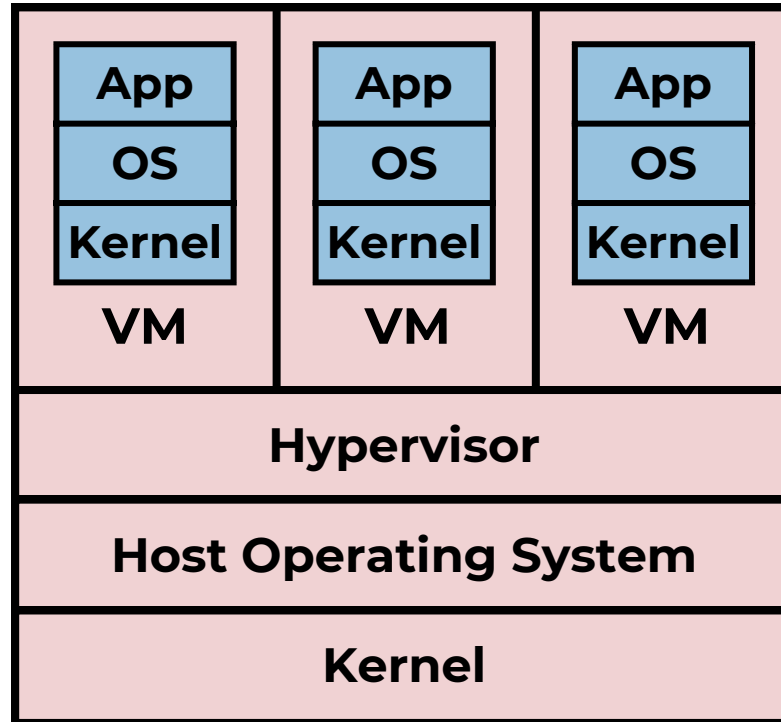
## Tipo 2



Exemplos: VMware, VirtualBox, KVM.

# Virtual Machines (VMs)

Tipo 2

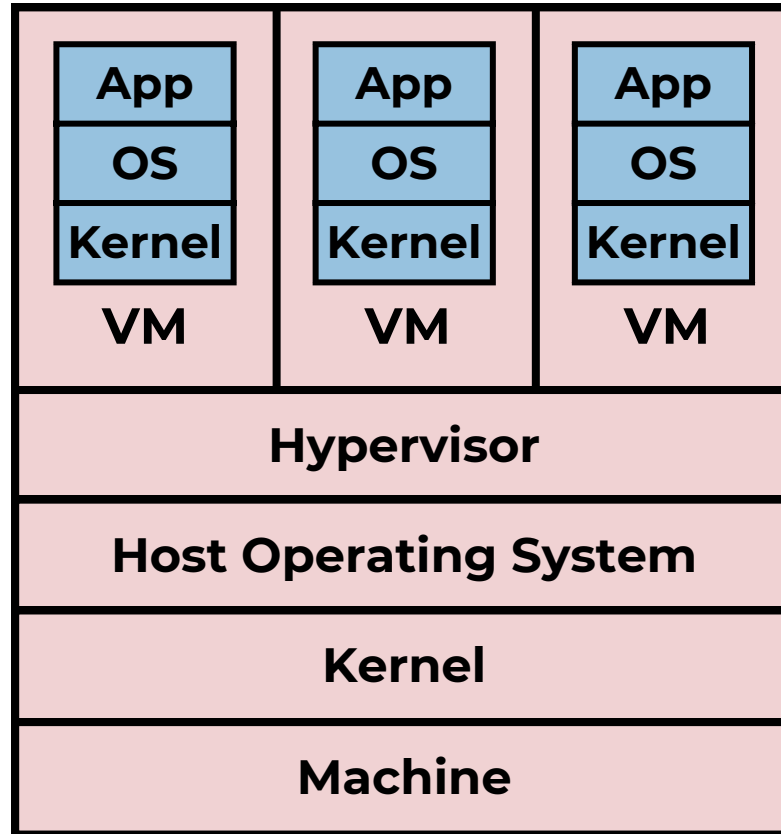


Exemplos: VMware, VirtualBox, KVM.



# Virtual Machines (VMs)

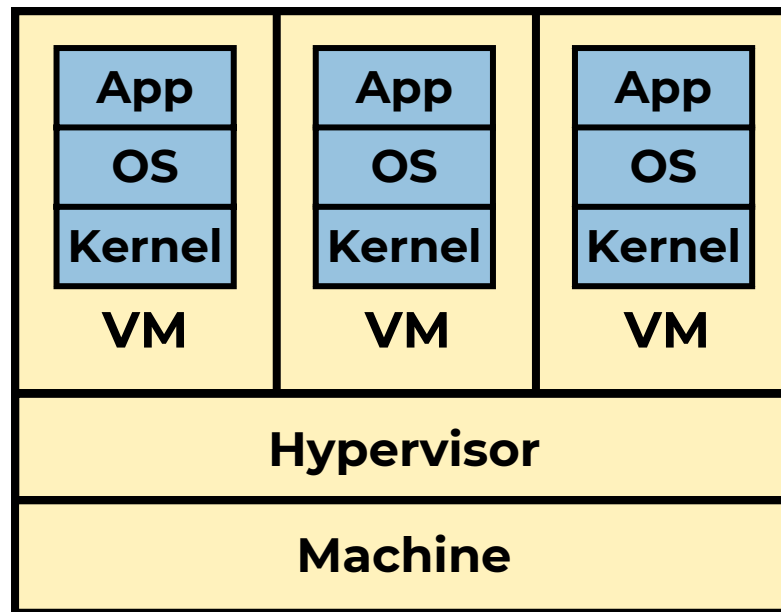
## Tipo 2



Exemplos: VMware, VirtualBox, KVM.

# Virtual Machines (VMs)

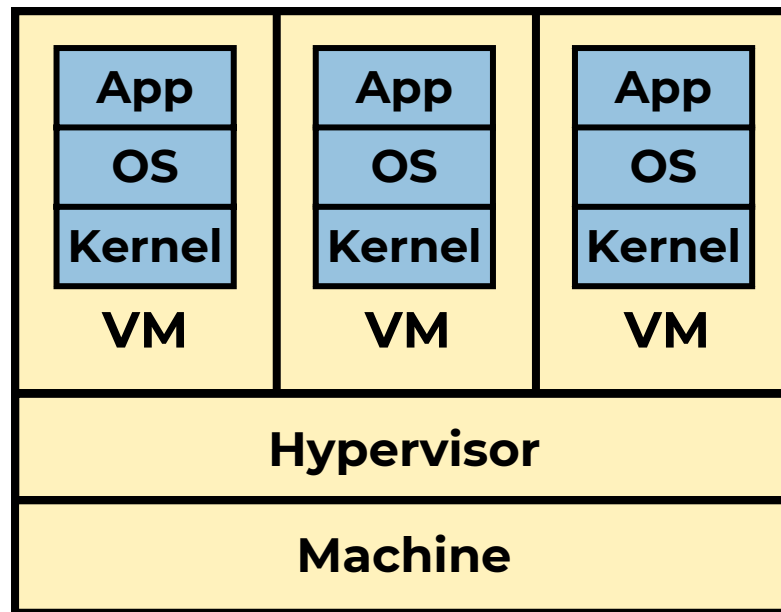
## Tipo 1



Exemplos: VMware ESXi, Microsoft Hyper-V, Xen.

# Virtual Machines (VMs)

Tipo 1

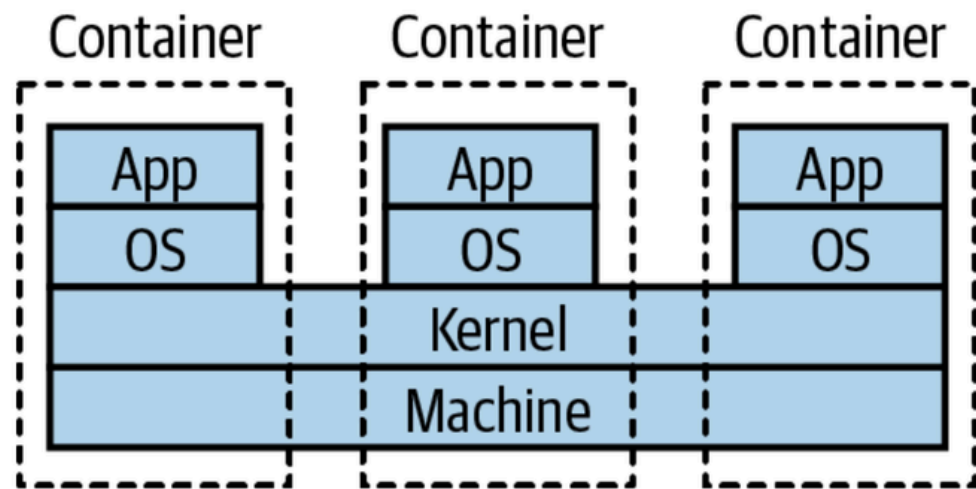


**WSL2: usa HyperV!**

Exemplos: VMware ESXi, Microsoft Hyper-V, Xen.

# Containers

- Virtualização
- Ambientes isolados para execução de aplicações
  - Todas as suas dependências (bibliotecas, configurações, etc.) para sua execução.
- **Kernel do sistema operacional host**



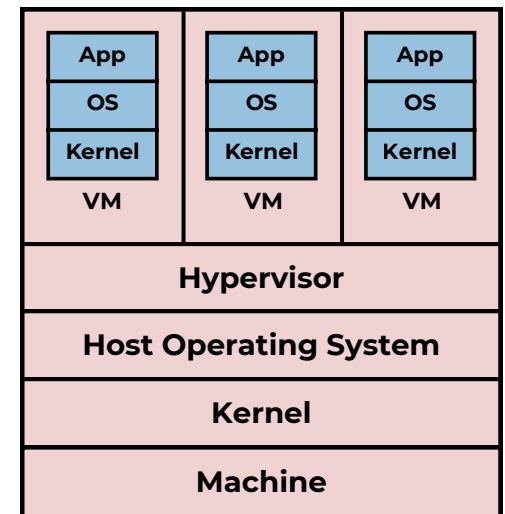
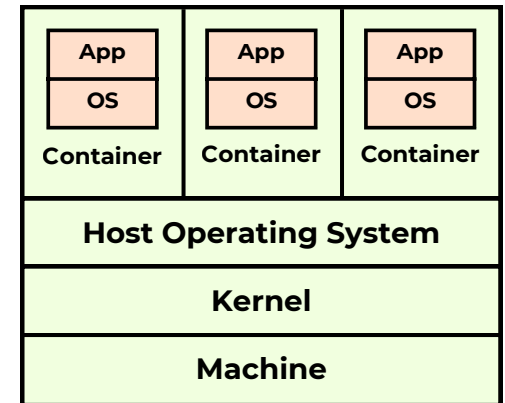
Exemplos: **Docker**, LXC.

# Containers

- O fato de ter somente o sistema operativo e a aplicação torna os containers muito leves, pequenos e portáteis
  - Microserviços
  - Serverless
  - Function-as-a-Service (FaaS)

# Comparação

- Containers:
  - Menor overhead
  - Inicialização rápida
  - Isolamento **dos processos**
  - Microserviços, CI/CD, e mais.
- Máquinas Virtuais:
  - Maior overhead
  - Inicialização lenta
  - Isolamento **completo do sistema operativo**
  - Ambientes *legacy* ou isolamento total.



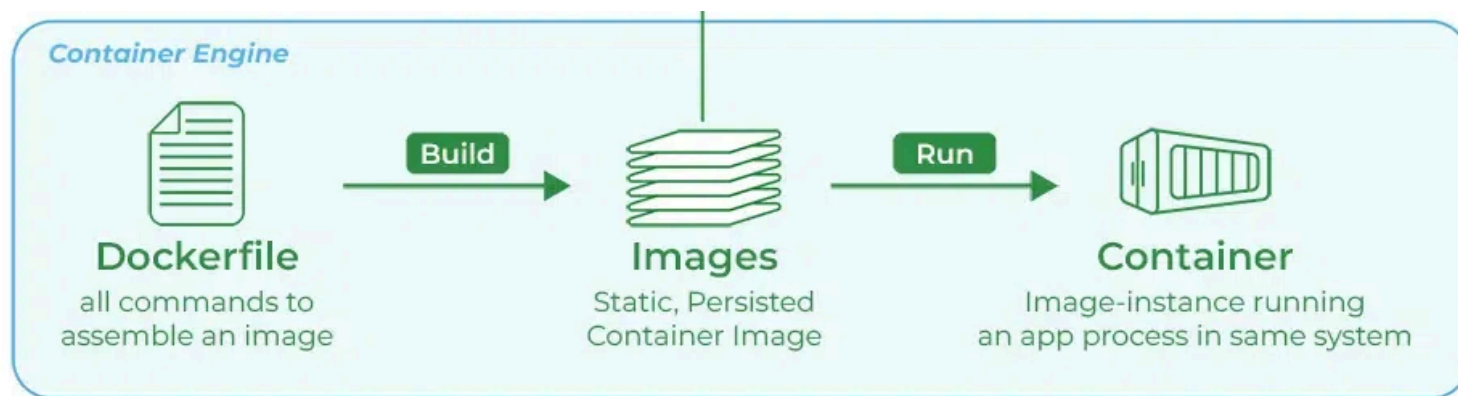
# Docker

- Principal plataforma de containerização usada atualmente
- Util para a criação, distribuição e execução de containers
- Portável, leve e robusto
- Utiliza **imagens** para criar os **containers**
- Docker Engine
  - Docker **Client**
    - Docker CLI
    - Docker Desktop
  - Docker **Host**
    - Docker Daemon (dockerd)



# Imagens e Containers

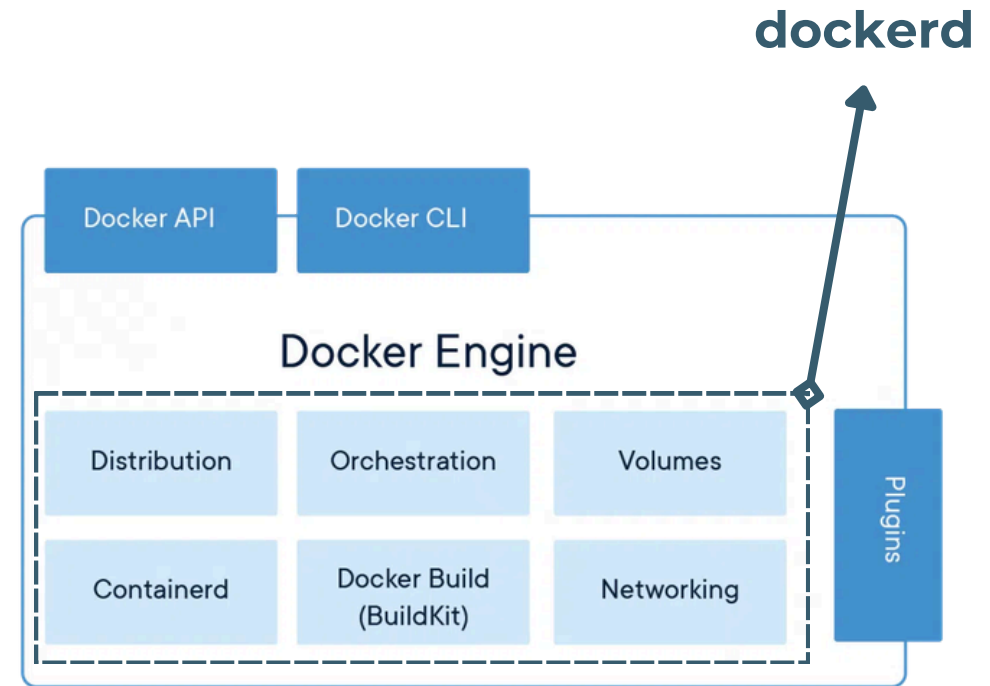
- **Dockerfile** contém todos os comandos necessários para que se crie uma imagem Docker.
- **Imagens** são modelos estáticos de como deverá ser criado o container
- **Container** é uma instanciação da imagem





# Docker Host

- Faz a gestão do ambiente de containers
- O Docker Daemon (dockerd) gerencia todas as operações do Docker.
  - Cria, executa, monitora e encerra containers
  - Gerencia imagens, volumes e redes.
  - ...

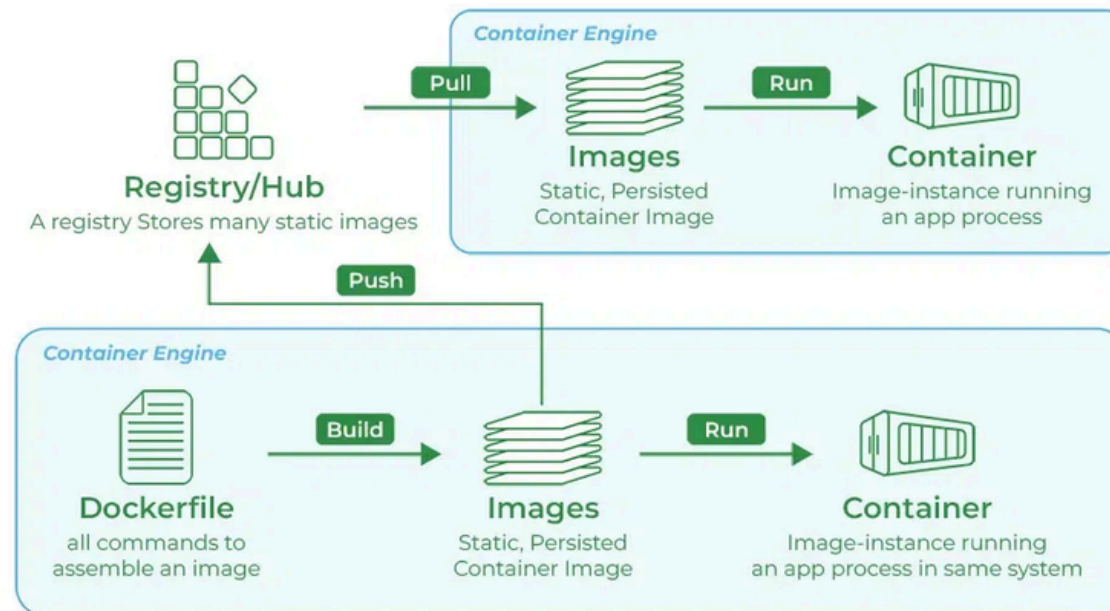


**Docker Client → Docker Daemon → executa os comandos**

# Docker Hub



- Serviço para armazenar, compartilhar e gerenciar imagens Docker.
- Basicamente qualquer imagem Docker que venha a ser necessária deve estar no Docker Hub.
- Repositórios públicos e privados

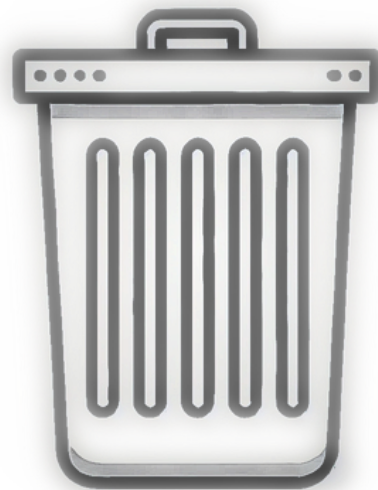


# Dados num container

- Containers são leves e portáteis.
  - Ótimos para serem inicializados e depois destruídos.
- O que acontece com os dados quando um container é destruído?

# Dados num container

- Containers são leves e portateis.
  - Ótimos para serem inicializados e depois destruídos.
- O que acontece com os dados quando um container é destruído?



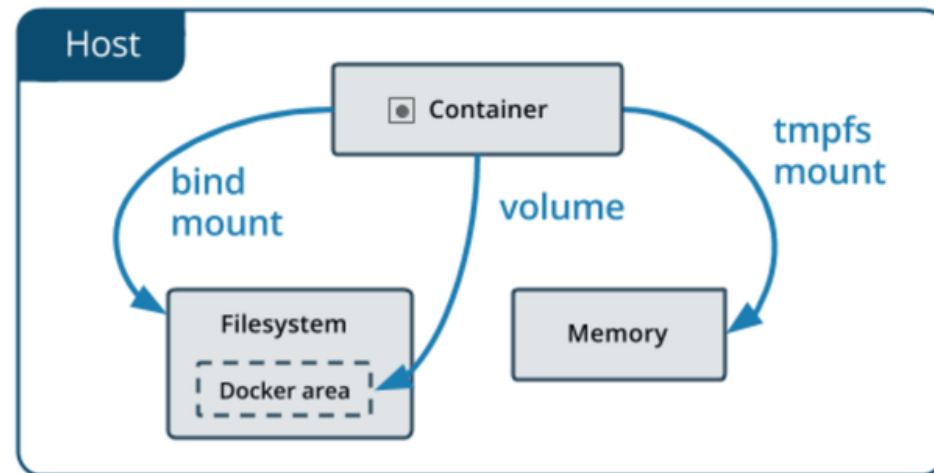
# Volumes e Bind Mounts

- **Volumes**

- Criados em locais específicos no host.
- Docker gere a localização e a integridade dos dados.
- Bons para dados que precisam ser compartilhados entre containers.

- **Bind Mounts**

- Montam um diretório (ou ficheiro) do host diretamente no container.
- Maior controle mas exige mais cuidados com permissões e organização.
- Bons para desenvolvimento (atualização em tempo real).

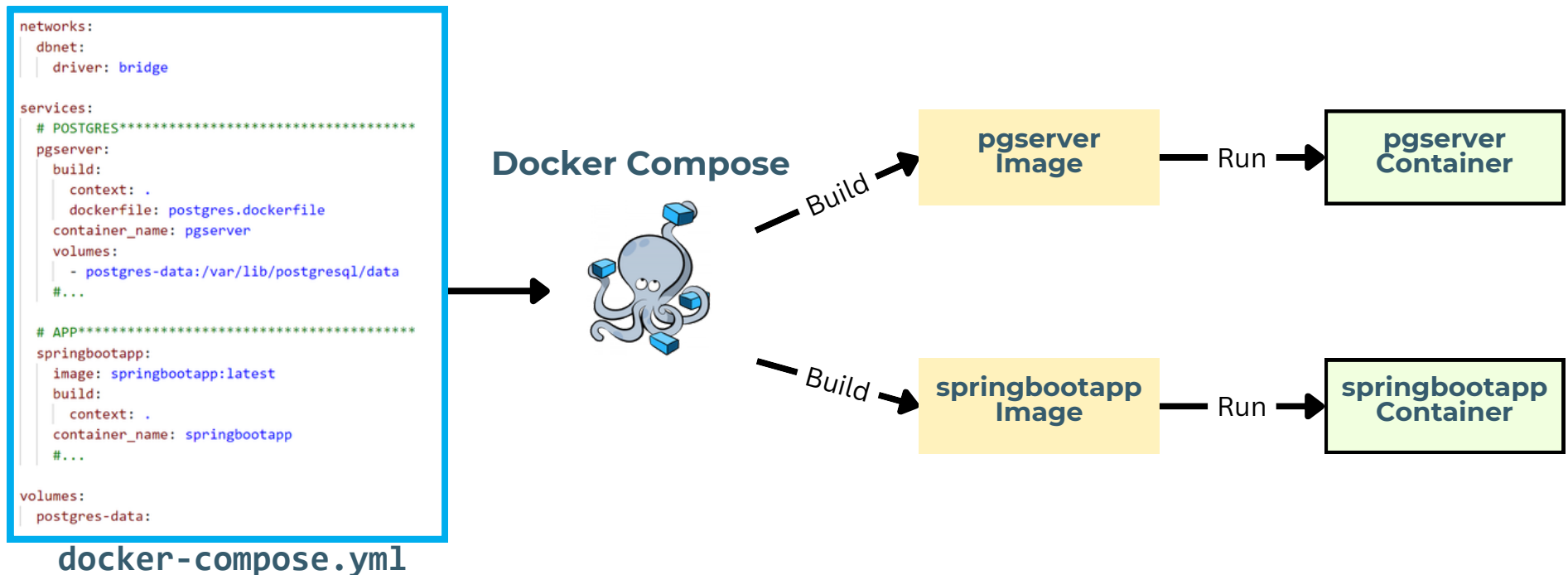


# Multiplos Containers

- Por muitas vezes, é necessário ter mais do que um container para que a aplicação funciona corretamente
  - Exemplo: Aplicação + Base de dados
- É possível lançar os containers separadamente contudo, faz mais sentido lança-los utilizando um único ficheiro de configuração
- **Docker Compose**

# Docker Compose

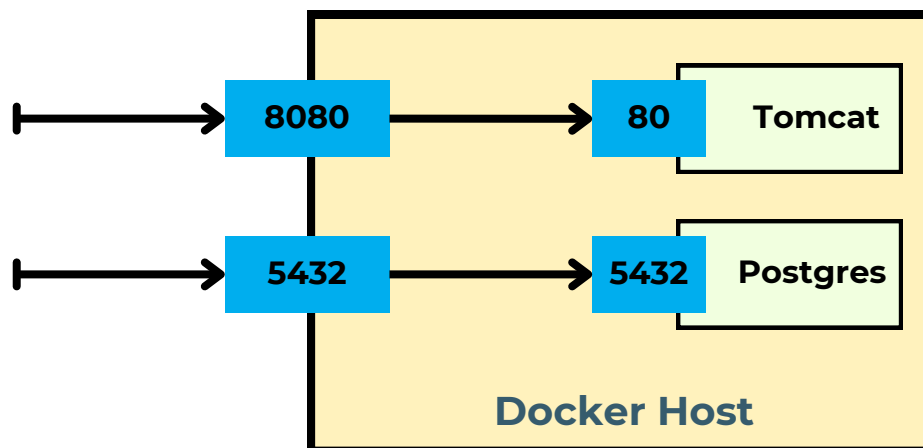
- Centraliza a configuração e garante consistência. Ficheiros YAML para configuração
  - Build, volumes, rede, portos, ...
- Facilita o deploy em diferentes ambientes (desenvolvimento, staging, produção).



# Multiplos Containers

## Comunicação

- Cada container tem seu próprio ambiente de rede isolado.
  - E.g., 127.0.0.1 (localhost) se refere ao container e não ao host.
- Ao definir uma rede com Docker, cada container pode ser acessado por seu nome
  - Serviço DNS interno mapeia os nomes para os IPs
- Para que um container possa ser acessado por outros, ele deve **expor os portos necessários**.
  - 8080:80 quer dizer que vamos mapear o porto 8080 do host para o porto 80 do container

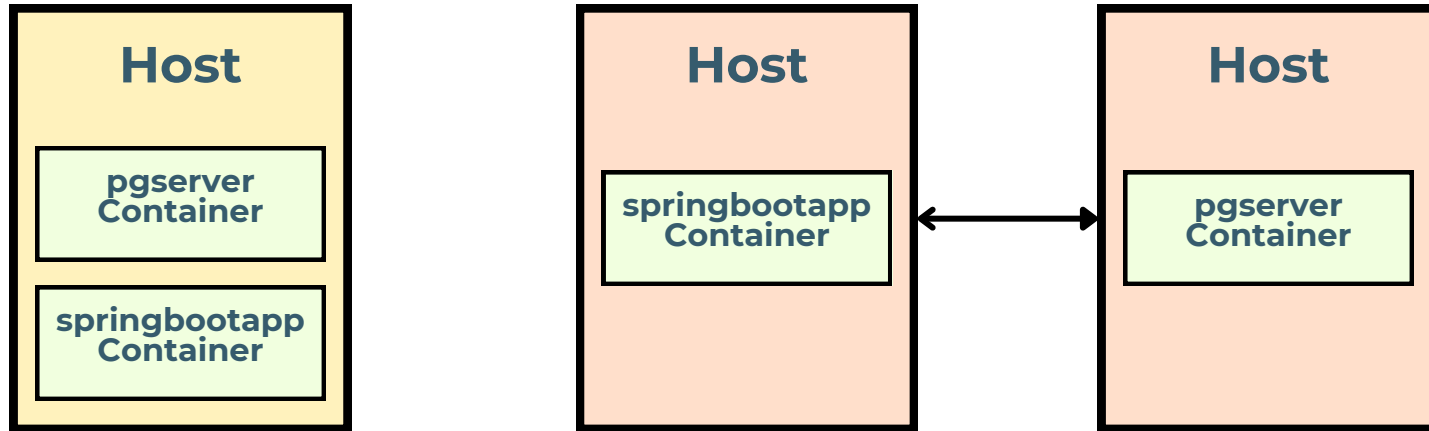




# Tipos de rede no Docker

- Bridge:
  - Rede padrão para containers.
  - Permite comunicação entre containers na mesma rede.
- Host:
  - O container compartilha a rede do host.
  - **Sem isolamento de rede.**
  - Elimina a necessidade de mapeamento de portas (utiliza diretamente a interface do host).
- Overlay:
  - Cria uma rede virtual que permite a comunicação entre containers em diferentes hosts.
  - Muito utilizado em ambientes de **orquestração**

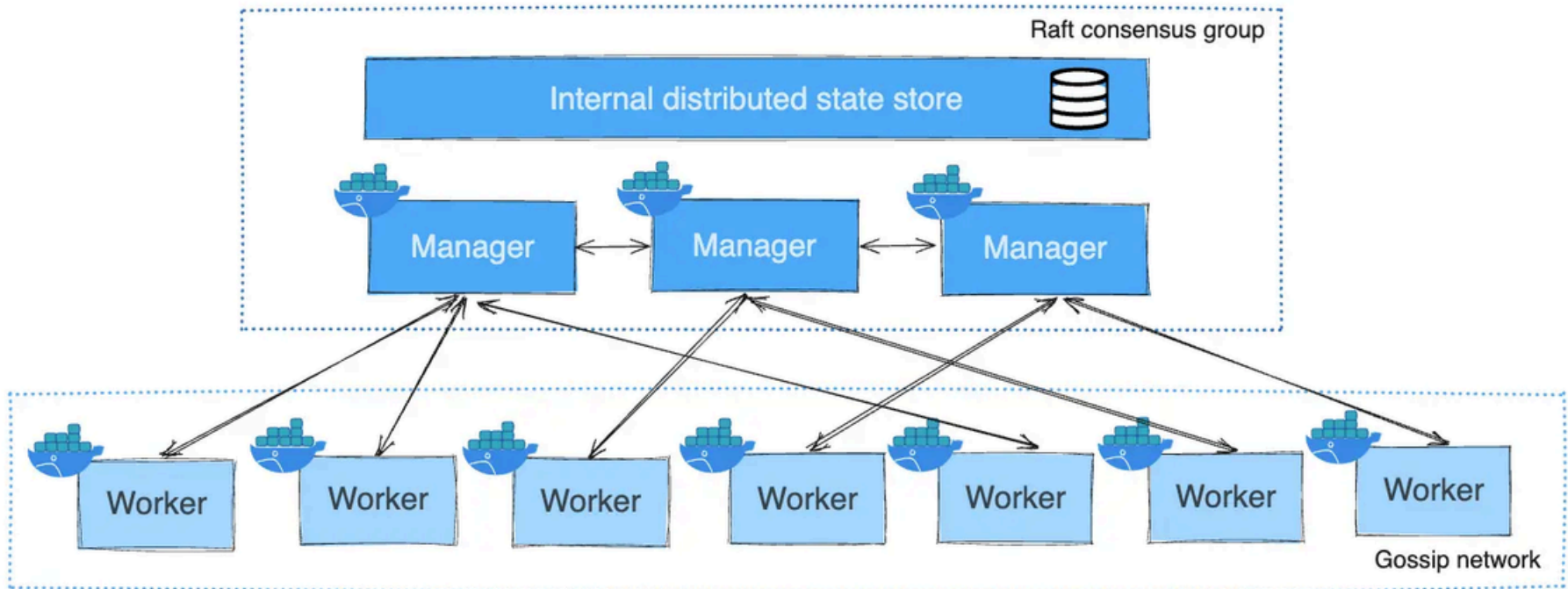
# Hosts único ou múltiplos hosts?

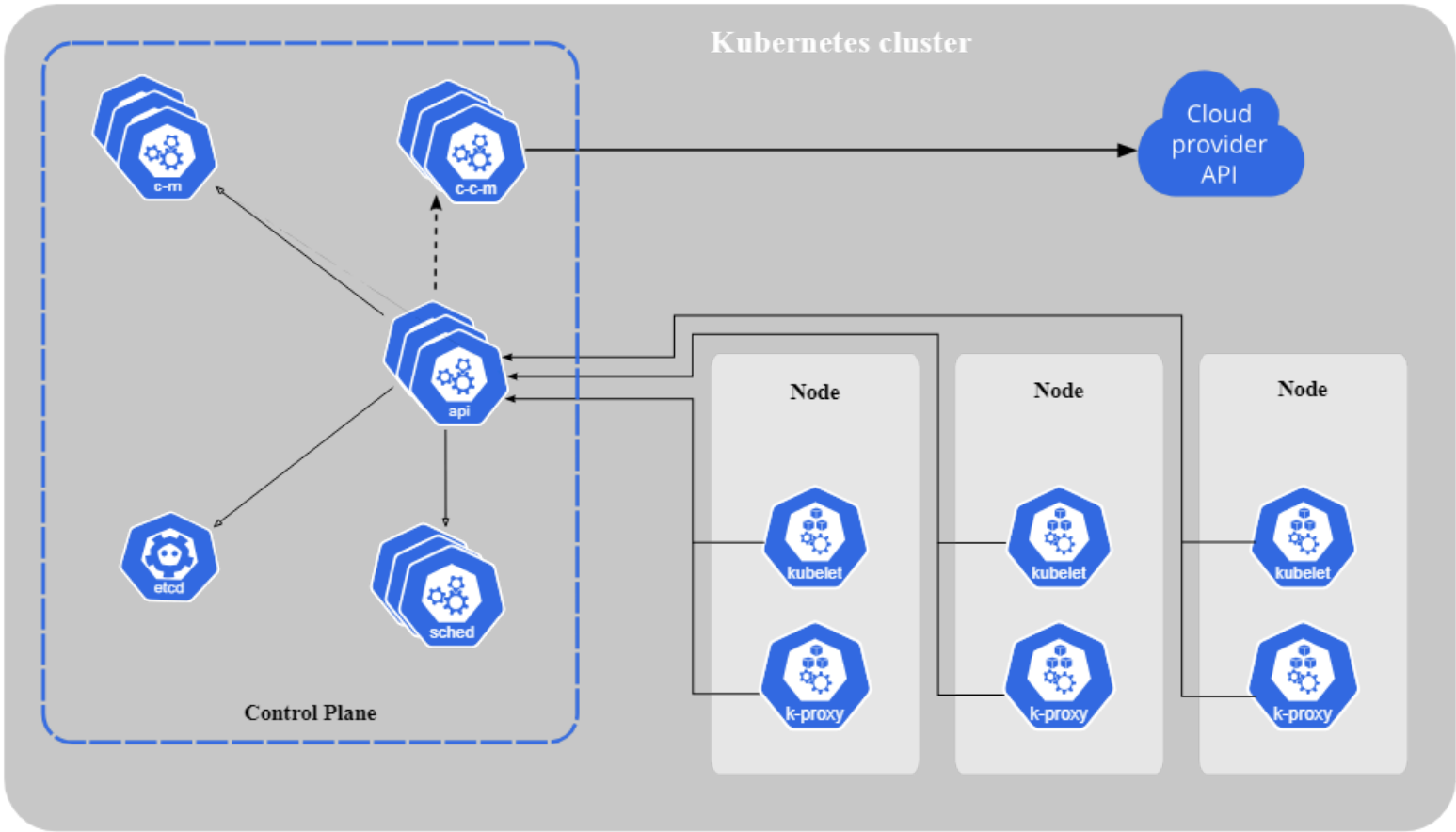


- Host único: cada serviço tem seu próprio container isolado.
- Múltiplos hosts: Cada serviço pode estar num host separado.
  - Faz uso de **orquestração** de containers para gerir os múltiplos hosts

# Orquestração de containers

- Processo de gerenciar, automatizar e coordenar a execução de containers.
  - Torna mais escalável, disponível, faz melhor uso dos recursos e cuida do ciclo de vida.
- Exemplos: **Kubernetes** e **Docker Swarm**
- Nomenclatura:
  - **Cluster:** conjunto de nodes
  - **Master Node:** gere o cluster + coordena atividades
  - **Worker Node:** executam os containers





- API server** 
- Cloud controller manager (optional)** 
- Controller manager** 
- etcd (persistence store)** 
- kubelet** 
- kube-proxy** 
- Scheduler** 
- Control plane** 
- Node** 

# Docker Swarm vs Kubernetes

- Kubernetes é mais adequado para projetos mais complexos
  - Melhor escalabilidade, recuperação automática mais robusta, melhor suporte da comunidade...
- Docker Swarm funciona como uma solução mais simples para projetos menos complexos
  - Mas que ainda demandam uma solução relativa a orquestração dos containers

# **Virtualização**

## **VMs, Containers e Docker**